

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ МАТЕМАТИКИ И ИНФОРМАЦИОННЫХ СИСТЕМ
ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ И ФИЗИКО-МАТЕМАТИЧЕСКИХ НАУК
КАФЕДРА ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

Допущен к защите

Заведующий кафедрой ПМИ

_____ Е.В. Разова

ПАРАЛЛЕЛЬНЫЕ АЛГОРИТМЫ УМНОЖЕНИЯ МАТРИЦ

Курсовой проект по дисциплине
«Проектная и научно-исследовательская деятельность»

Выполнил студент группы ПМИб-2301-52-00 _____ / П.В. Рылова /

Руководитель к.пед.н., доцент кафедры ПМИ _____ / А.Н. Соколова/

Работа защищена с оценкой _____ _____.2025 г.

Члены комиссии: _____ / _____ /
_____ / _____ /

Содержание

Введение.....	3
1 Теоретические основы умножения матриц и параллельных алгоритмов..	5
1.1 Постановка задачи и основные определения.....	5
1.1.1 Постановка задачи.....	5
1.1.2 Основные определения.....	5
1.2 Вычислительная сложность умножения матриц.....	6
1.2.1 Классификация алгоритмов	6
1.3.2. Преимущества и недостатки различных подходов	7
1.3 Параллельные алгоритмы умножения матриц	8
1.4 Выводы по разделу 1	19
2 Программная реализация параллельных алгоритмов умножения матриц	20
2.1 Технологии параллельного программирования	20
2.2 Выбор языка и среды программирования.....	21
2.3 Проектирование пользовательского интерфейса.....	22
2.4 Реализованные функции и алгоритмы	23
2.5 Экспериментальное сравнение параллельных алгоритмов умножения матриц	24
2.6 Выводы по разделу 2.....	25
Заключение	29
Библиографический список	30
Приложения	32
Приложение А. Алгоритм умножения матриц при ленточной схеме разделения данных.....	32
Приложение Б. Алгоритм Фокса умножения матриц при блочном разделении данных.....	32
Приложение В. Алгоритм Кэннона умножения матриц при блочном разделении данных.....	45

Введение

Умножение матриц – один из базовых алгоритмов, который широко применяется в различных численных методах, в частности, в алгоритмах машинного обучения (логическая регрессия, методы опорных векторов (SVM), нейронные сети, методы ансамблей), в обработке изображений (фильтрация, преобразование Фурье, поворот изображений), в графике и 3D-моделировании (в играх и анимации необходимо часто менять положение, размер и ориентацию моделей – все это представляется матрицей). Для матриц большой размерности временные затраты на выполнение умножения растут, поэтому актуальным является разработка и применение оптимальных алгоритмов, выполняющих матричное умножение быстро. Одним из таких подходов является распараллеливание умножения матриц и выполнение на нескольких процессорах.

Проблема исследования заключается в отсутствии рекомендации по применению параллельных алгоритмов матричного умножения.

Целью курсового проекта является исследование возможностей и ограничений параллельных алгоритмов матричного умножения.

Для достижения поставленной цели необходимо решить следующие **задачи**:

выполнить обзор научно-технической литературы по вопросам параллельных алгоритмов матричного умножения.

реализовать на языке программирования параллельные алгоритмы матричного умножения.

подготовить данные для тестирования алгоритмов.

провести экспериментальное исследование реализованных алгоритмов и сформулировать рекомендации по применению параллельных алгоритмов матричного умножения.

Курсовой проект состоит из введения, двух разделов, заключения, библиографического списка и приложения с листингом программы. В первом

разделе рассматривается теория матричного умножения, включая основные определения и алгоритмы. Также обсуждается вычислительная сложность различных подходов к умножению матриц и важность параллельных алгоритмов для ускорения вычислений на больших матрицах. Во втором разделе описываются технологии параллельного программирования, выбор языка и среды для реализации, а также сами алгоритмы. Проводится экспериментальное исследование для сравнения производительности различных параллельных методов матричного умножения.

1 Теоретические основы умножения матриц и параллельных алгоритмов

1.1 Постановка задачи и основные определения

1.1.1 Постановка задачи

Задача умножения матриц заключается в вычислении матрицы C – произведения двух матриц A и B параллельными алгоритмами.

1.1.2 Основные определения

Матрица – это прямоугольная таблица чисел, организованных в строки и столбцы. Элемент матрицы A обозначается как A_{ij} , где i – номер строки, j – номер столбца.

Умножение матриц – это фундаментальная операция линейной алгебры, которая находит широкое применение в различных областях науки и техники.

Пусть даны две матрицы: $A = (A_{ij})$ размерности $m \times n$ и $B = (B_{ik})$ размерности $n \times p$, тогда их произведение $C = A \times B$ – это матрица размерности $m \times p$, элементы которой вычисляются по формуле:

$$C_{ik} = \sum_{j=1}^n A_{ij} B_{jk} \quad 1 \leq i \leq m, \quad 1 \leq k \leq p,$$

где

- A_{ij} – элементы матрицы A ,
- B_{ik} – элементы матрицы B ,
- C_{ik} – элементы результирующей матрицы C .

1.2 Вычислительная сложность умножения матриц

1.2.1 Классификация алгоритмов

Алгоритмы умножения матриц можно классифицировать по следующим критериям:

1. **Классический алгоритм** – прямой метод с вычислительной сложностью $O(n^3)$ [Миллер, Уэбб (1975), «Вычислительная сложность и численная стабильность», SIAM News].
2. **Оптимизированные последовательные алгоритмы** – алгоритмы, снижающие вычислительную сложность (например, алгоритм Штрассена $O(n^{2.81})$ [Миллер, Уэбб (1975), «Вычислительная сложность и численная стабильность», SIAM News], Алгоритмы Копперсмита-Винограда $O(n^{2.3728596})$ [Миллер, Уэбб (1975), «Вычислительная сложность и численная стабильность», SIAM News]).
3. **Параллельные алгоритмы** – распределенные вычисления между несколькими процессорами или потоками с вычислительной сложностью $O(\frac{n^3}{p})$ [Миллер, Уэбб (1975), «Вычислительная сложность и численная стабильность», SIAM News].

1.3.2. Преимущества и недостатки различных подходов

- **Классический алгоритм**
 - **Преимущества:** простая реализация, легко понять и реализовать.
 - **Недостатки:** долгое выполнение при больших размерах матриц из-за вычислительной сложности.
- **Оптимизированные последовательные алгоритмы**
 - **Преимущества:** позволяют сократить время вычислений за счет уменьшения количества операций.
 - **Недостатки:** усложняются алгоритмические структуры, требуется больше памяти и вычислительных ресурсов для рекурсий.
- **Параллельные алгоритмы**
 - **Преимущества:** значительное ускорение вычислений за счет распределения нагрузки на несколько процессоров.
 - **Недостатки:** требуется грамотное управление потоками и балансировка нагрузки, возможны дополнительные накладные расходы на передачу данных между процессами.

Данный проект посвящен исследованию параллельных алгоритмы, так как они обеспечивают значительное увеличение производительности при больших размерах входных данных. Параллельные алгоритмы позволяют эффективно использовать многопоточные и многопроцессорные вычислительные системы. Их применение особенно актуально для задач машинного обучения, графической обработки и моделирования физических процессов.

Параллельные алгоритмы умножения матриц

1.3.1 Умножение матриц при ленточной схеме разделения данных

Историческая справка

Ленточная схема разделения данных используется с 1960-х годов в параллельных вычислениях для эффективного распределения матриц между процессами

[<https://www.sciencedirect.com/science/article/abs/pii/S0965997806000147>]. Она хорошо подходит для систем с линейной топологией процессов и позволяет параллельно умножать матрицы, разбивая данные по строкам и столбцам.

Математическое описание

Пусть A, B – квадратные матрицы порядка n , а $C = A \times B$. Пусть p – количество процессов. При ленточной схеме:

- матрица A делится по строкам, т.е. каждый процесс получает блок строк A_i , $0 \leq i \leq n$.
- Матрица B делится по столбцам, т.е. есть каждый процесс получает блок столбцов B_j , $0 \leq j \leq n$.

Каждый процесс вычисляет подматрицу:

$$C_{ij} = A_i \times B_j.$$

Затем данные суммируются или пересылаются для формирования полного результата (рис. 1.1).

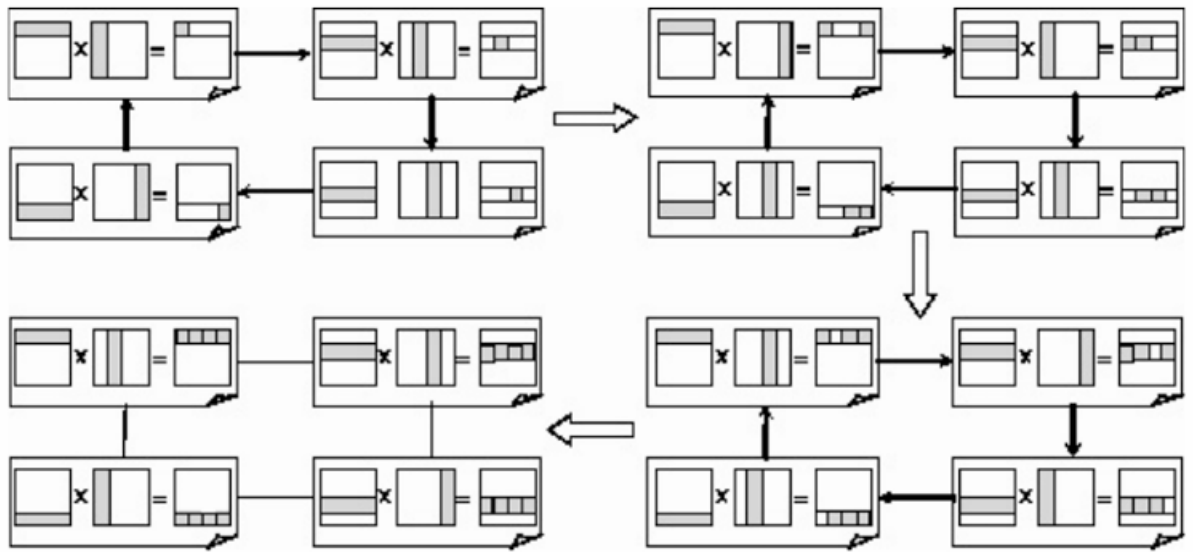


Рисунок 1.1 – Процесс вычисления подматриц

Алгоритм

Разбить A по строкам и B по столбцам на p полос. Для каждого процесса:

1. Получить строку A_i и столбец B_j
2. Для $i, j = \{0, 1, 2, \dots, p-1\}$:
 - Обмен полос B по кольцу.
 - Выполнить $C_{ij} += A_i \times B_j$.

Временная сложность

- последовательное умножение $O(n^3)$ [Миллер, Уэбб (1975), «Вычислительная сложность и численная стабильность», SIAM News];
- параллельная версия (Фокс, p процессоров) $O(n^3/p)$ [Миллер, Уэбб (1975), «Вычислительная сложность и численная стабильность», SIAM News].

Пример реализации алгоритма

Допустим, у нас есть две матрицы A и B размерности 4×4 :

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{pmatrix}, \quad B = \begin{pmatrix} 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 \\ 25 & 26 & 27 & 28 \\ 29 & 30 & 31 & 32 \end{pmatrix}.$$

Вычисляем каждый элемент матрицы C :

- $C_{00} = A_0 \times B_0 = 1 * 17 + 2 * 21 + 3 * 25 + 4 * 29 = 250$
- $C_{01} = A_0 \times B_1 = 1 * 18 + 2 * 22 + 3 * 26 + 4 * 30 = 260$
- $C_{02} = A_0 \times B_2 = 1 * 19 + 2 * 23 + 3 * 27 + 4 * 31 = 214$
- $C_{03} = A_0 \times B_3 = 1 * 20 + 2 * 24 + 3 * 28 + 4 * 32 = 224$
- $C_{10} = A_1 \times B_0 = 5 * 17 + 6 * 21 + 7 * 25 + 8 * 29 = 618$
- $C_{11} = A_1 \times B_1 = 5 * 18 + 6 * 22 + 7 * 26 + 8 * 30 = 644$
- $C_{12} = A_1 \times B_2 = 5 * 19 + 6 * 23 + 7 * 27 + 8 * 31 = 526$
- $C_{13} = A_1 \times B_3 = 5 * 20 + 6 * 24 + 7 * 28 + 8 * 32 = 552$
- $C_{20} = A_2 \times B_0 = 9 * 17 + 2 * 21 + 3 * 25 + 12 * 29 = 1184$
- $C_{21} = A_2 \times B_1 = 9 * 18 + 2 * 22 + 3 * 26 + 12 * 30 = 1226$
- $C_{22} = A_2 \times B_2 = 9 * 19 + 2 * 23 + 3 * 27 + 12 * 31 = 1270$
- $C_{23} = A_2 \times B_3 = 9 * 20 + 2 * 24 + 3 * 28 + 12 * 32 = 1312$
- $C_{30} = A_3 \times B_0 = 13 * 17 + 2 * 21 + 3 * 25 + 16 * 29 = 1632$
- $C_{31} = A_3 \times B_1 = 13 * 18 + 2 * 22 + 3 * 26 + 16 * 30 = 1690$
- $C_{32} = A_3 \times B_2 = 13 * 19 + 2 * 23 + 3 * 27 + 16 * 31 = 1752$
- $C_{33} = A_3 \times B_3 = 13 * 20 + 2 * 24 + 3 * 28 + 16 * 32 = 1810$

Результат собирается из блоков $C_{00}, C_{01}, \dots, C_{33}$. Итоговая матрица имеет вид

$$C = \begin{pmatrix} 250 & 260 & 214 & 224 \\ 618 & 644 & 526 & 552 \\ 1184 & 1226 & 1270 & 1312 \\ 1632 & 1690 & 1752 & 1810 \end{pmatrix}.$$

1.3.2. Алгоритм Фокса умножения матриц при блочном разделении данных

История

Алгоритм Фокса был предложен в 1973 году Линдси Фоксом для эффективного параллельного умножения матриц [https://www.sciencedirect.com/science/article/abs/pii/S0965997806000147]. Он разработан для работы на многопроцессорных системах с топологией двумерной сетки и является улучшением алгоритма Кэннона, уменьшая объем пересылки данных.

Математическое описание

Пусть все матрицы квадратные размера $n \times n$. Матрицы разбиваются на $q \times q$ блоков размера $k \times k$, где $k = n/q$. Тогда операция умножения матриц A и B может быть представлена в блочной форме:

$$\begin{pmatrix} A_{00} & A_{01} & \dots & A_{0q-1} \\ & & \dots & \\ A_{q-10} & A_{q-11} & \dots & A_{q-1q-1} \end{pmatrix} \times \begin{pmatrix} B_{00} & B_{01} & \dots & B_{0q-1} \\ & & \dots & \\ B_{q-10} & B_{q-11} & \dots & B_{q-1q-1} \end{pmatrix} \\ = \begin{pmatrix} C_{00} & C_{01} & \dots & C_{0q-1} \\ & & \dots & \\ C_{q-10} & C_{q-11} & \dots & C_{q-1q-1} \end{pmatrix},$$

где каждый блок C_{ij} матрицы C определяется в соответствии с выражением

$$C_{ij} = \sum_{s=0}^{q-1} A_{is} B_{js}.$$

Алгоритм

Разбиение данных:

1. Матрицы A , B и C разделяются на блоки размером $\frac{n}{q} \times \frac{n}{q}$;
2. Каждый процессор P_{ij} получает свою подматрицу A_{ij} и B_{ij} .

Основные шаги алгоритма

Для каждого шага k (от 0 до $q - 1$):

1. Передача блоков матрицы A . Каждый процессор P_{ij} получает блок $A_{i,(j+i-k) \bmod q}$ из своей строки.

2. Вычисление локального произведения. Каждым процессором выполняется умножение полученного блока A на текущий блок B_{ij} .

3. Сдвиг блоков B вверх на одну строку. Каждый процессор P_{ij} отправляет свой блок B_{ij} процессору $P_{(i-1) \bmod q, j}$.

4. Повторение шагов для всех k (количество блоков).

Временная сложность

- Последовательное умножение $O(n^3)$ [Миллер, Уэбб (1975), «Вычислительная сложность и численная стабильность», SIAM News];
- Параллельная версия (Фокс, p процессоров) $O(n^3/p)$ [Миллер, Уэбб (1975), «Вычислительная сложность и численная стабильность», SIAM News].

Пример реализации алгоритма

Допустим, у нас есть две матрицы A и B размером 4×4

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{pmatrix} \quad B = \begin{pmatrix} 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 \\ 25 & 26 & 27 & 28 \\ 29 & 30 & 31 & 32 \end{pmatrix}$$

Разделим на блоки 2×2

$$A_{00} = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} \quad B_{00} = \begin{bmatrix} 17 & 18 \\ 21 & 22 \end{bmatrix}$$

$$A_{01} = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \quad B_{01} = \begin{bmatrix} 19 & 20 \\ 23 & 24 \end{bmatrix}$$

$$A_{10} = \begin{bmatrix} 9 & 10 \\ 13 & 14 \end{bmatrix} \quad B_{10} = \begin{bmatrix} 25 & 26 \\ 29 & 30 \end{bmatrix}$$

$$A_{11} = \begin{bmatrix} 11 & 12 \\ 15 & 16 \end{bmatrix} \quad B_{11} = \begin{bmatrix} 27 & 28 \\ 31 & 32 \end{bmatrix}$$

Итерация 1 ($k = 0$)

Выбор A :

- C_{00} и C_{01} берут A_{00}
- C_{10} и C_{11} берут A_{10}

Умножения:

$$C_{00}: A_{00} \times B_{00}$$

$$\begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} \times \begin{bmatrix} 17 & 18 \\ 21 & 22 \end{bmatrix} = \begin{bmatrix} 59 & 62 \\ 211 & 222 \end{bmatrix}$$

$$C_{01}: A_{00} \times B_{01}$$

$$\begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} \times \begin{bmatrix} 19 & 20 \\ 23 & 24 \end{bmatrix} = \begin{bmatrix} 65 & 68 \\ 233 & 224 \end{bmatrix}$$

$$C_{10}: A_{10} \times B_{10}$$

$$\begin{bmatrix} 9 & 10 \\ 13 & 14 \end{bmatrix} \times \begin{bmatrix} 25 & 26 \\ 29 & 30 \end{bmatrix} = \begin{bmatrix} 515 & 534 \\ 731 & 758 \end{bmatrix}$$

$$C_{11}: A_{10} \times B_{11}$$

$$\begin{bmatrix} 9 & 10 \\ 13 & 14 \end{bmatrix} \times \begin{bmatrix} 27 & 28 \\ 31 & 32 \end{bmatrix} = \begin{bmatrix} 545 & 564 \\ 775 & 802 \end{bmatrix}$$

Итерация 2 ($k = 1$)

Выбор A:

- C_{00} и C_{01} берут A_{01}
- C_{10} и C_{11} берут A_{11}

Умножения:

$$C_{00}: A_{01} \times B_{10}$$

$$\begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \times \begin{bmatrix} 25 & 26 \\ 29 & 30 \end{bmatrix} = \begin{bmatrix} 191 & 198 \\ 407 & 422 \end{bmatrix}$$

$$C_{01}: A_{01} \times B_{11}$$

$$\begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \times \begin{bmatrix} 27 & 28 \\ 31 & 32 \end{bmatrix} = \begin{bmatrix} 149 & 156 \\ 293 & 308 \end{bmatrix}$$

$$C_{10}: A_{11} \times B_{00}$$

$$\begin{bmatrix} 11 & 12 \\ 15 & 16 \end{bmatrix} \times \begin{bmatrix} 17 & 18 \\ 21 & 22 \end{bmatrix} = \begin{bmatrix} 669 & 692 \\ 901 & 932 \end{bmatrix}$$

$$C_{11}: A_{11} \times B_{01}$$

$$\begin{bmatrix} 11 & 12 \\ 15 & 16 \end{bmatrix} \times \begin{bmatrix} 19 & 20 \\ 23 & 24 \end{bmatrix} = \begin{bmatrix} 725 & 748 \\ 977 & 1008 \end{bmatrix}$$

Сложим C_{ij} при $i = \{0,1\}$ и $j = \{0,1\}$ итерации 1 к C_{nm} при $n = \{0,1\}$ и $m = \{0,1\}$ итерации 2. Тогда получаем

$$C_{00} = \begin{bmatrix} 59 & 62 \\ 211 & 222 \end{bmatrix} + \begin{bmatrix} 191 & 198 \\ 407 & 422 \end{bmatrix} = \begin{bmatrix} 250 & 260 \\ 618 & 644 \end{bmatrix}$$

$$C_{01} = \begin{bmatrix} 65 & 68 \\ 233 & 224 \end{bmatrix} + \begin{bmatrix} 149 & 156 \\ 293 & 308 \end{bmatrix} = \begin{bmatrix} 214 & 224 \\ 526 & 552 \end{bmatrix}$$

$$C_{10} = \begin{bmatrix} 515 & 534 \\ 731 & 758 \end{bmatrix} + \begin{bmatrix} 669 & 692 \\ 901 & 932 \end{bmatrix} = \begin{bmatrix} 1184 & 1226 \\ 1632 & 1690 \end{bmatrix}$$

$$C_{11} = \begin{bmatrix} 545 & 564 \\ 775 & 802 \end{bmatrix} + \begin{bmatrix} 725 & 748 \\ 977 & 1008 \end{bmatrix} = \begin{bmatrix} 1270 & 1312 \\ 1752 & 1810 \end{bmatrix}$$

Результат собирается из блоков $C_{00}, C_{01}, C_{10}, C_{11}$. Итог

$$C = \begin{pmatrix} 250 & 260 & 214 & 224 \\ 618 & 644 & 526 & 552 \\ 1184 & 1226 & 1270 & 1312 \\ 1632 & 1690 & 1752 & 1810 \end{pmatrix}.$$

1.3.3. Алгоритм Кэннона умножения матриц при блочном разделении данных

История

Алгоритм Кэннона (Cannon's Algorithm) был предложен в 1969 году для эффективного параллельного умножения квадратных матриц [<https://www.sciencedirect.com/science/article/abs/pii/S0965997806000147>]. Он минимизирует объём передачи данных между процессами и эффективно работает на квадратной сетке процессов размером $q \times q$, где $p = q^2$ – общее число процессов.

Математическое описание

Пусть A, B - входные квадратные матрицы. Мы делим их на $q \times q$ блоков размером $k \times k$, где $k = n/q$. Таким образом, каждая матрица разбита на одинаковые квадратные подматрицы.

Каждый процесс P_{ij} хранит:

- Блок A_{ij} – элемент из матрицы A с позицией (i, j) ;
- блок B – аналогичный блок из B ;
- блок результата C , который инициализируется нулями.

Перед началом вычислений:

- Каждый блок A_{ij} сдвигается влево по строке на i позиций:

$$A_{ij} \rightarrow A_{i, (j+i) \bmod q}$$

- Каждый блок B_{ij} сдвигается вверх по столбцу на j позиций:

$$B_{ij} \rightarrow B_{(i+j), j \bmod q}$$

Эти сдвиги нужны, чтобы на первой итерации у каждого процесса уже были «правильные» блоки для начала вычислений.

Основной цикл вычислений (всего q шагов)

На каждой из q итераций:

1. Каждый процесс P_{ij} умножает текущие блоки $A_{ij} \times B_{ij}$ и добавляет результат к C_{ij} :

$$C_{ij} += A_{ij} \times B_{ij}$$

2. Затем:

- блок A_{ij} сдвигается влево на 1 позицию в строке i ;
- блок B_{ij} сдвигается вверх на 1 позицию в столбце j

Таким образом, каждый процесс получает новые блоки на следующей итерации.

Алгоритм

Разбить матрицы A, B, C на блоки $A[i][j], B[i][j], C[i][j]$

Сдвинуть $A[i][j]$ влево на i позиций

Сдвинуть $B[i][j]$ вверх на j позиций

Для $k = 0$ до $q - 1$:

$$C[i][j] += A[i][j] \times B[i][j]$$

$$A[i][j] \leftarrow A[i][(j - 1) \bmod q] - \text{сдвиг } A \text{ влево}$$

$$B[i][j] \leftarrow B[(i - 1) \bmod q][j] - \text{сдвиг } B \text{ вверх}$$

Временная сложность

- Последовательное умножение $O(n^3)$ [Миллер, Уэбб (1975), «Вычислительная сложность и численная стабильность», SIAM News];
- Параллельная версия (Фокс, p процессоров) $O(n^3/p^2)$ при $p = q^2$ процессах [Миллер, Уэбб (1975), «Вычислительная сложность и численная стабильность», SIAM News].

Пример реализации алгоритма

Допустим, у нас есть две матрицы A и B размером 4×4

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{pmatrix}, \quad B = \begin{pmatrix} 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 \\ 25 & 26 & 27 & 28 \\ 29 & 30 & 31 & 32 \end{pmatrix}.$$

Разделим на блоки размерности 2×2 :

$$A_{00} = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} \quad B_{00} = \begin{bmatrix} 17 & 18 \\ 21 & 22 \end{bmatrix}$$

$$A_{01} = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \quad B_{01} = \begin{bmatrix} 19 & 20 \\ 23 & 24 \end{bmatrix}$$

$$A_{10} = \begin{bmatrix} 9 & 10 \\ 13 & 14 \end{bmatrix} \quad B_{10} = \begin{bmatrix} 25 & 26 \\ 29 & 30 \end{bmatrix}$$

$$A_{11} = \begin{bmatrix} 11 & 12 \\ 15 & 16 \end{bmatrix} \quad B_{11} = \begin{bmatrix} 27 & 28 \\ 31 & 32 \end{bmatrix}$$

Итерация 1 ($k = 0$)

Выбор A:

- C_{00} и C_{01} берут A_{00}
- C_{10} и C_{11} берут A_{10}

Умножения:

$$C_{00}^0: A_{00} \times B_{00}$$

$$\begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} \times \begin{bmatrix} 17 & 18 \\ 21 & 22 \end{bmatrix} = \begin{bmatrix} 59 & 62 \\ 211 & 222 \end{bmatrix}$$

$$C_{01}^0: A_{00} \times B_{01}$$

$$\begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} \times \begin{bmatrix} 19 & 20 \\ 23 & 24 \end{bmatrix} = \begin{bmatrix} 65 & 68 \\ 233 & 224 \end{bmatrix}$$

$$C_{10}^0: A_{10} \times B_{10}$$

$$\begin{bmatrix} 9 & 10 \\ 13 & 14 \end{bmatrix} \times \begin{bmatrix} 25 & 26 \\ 29 & 30 \end{bmatrix} = \begin{bmatrix} 515 & 534 \\ 731 & 758 \end{bmatrix}$$

$$C_{11}^0: A_{10} \times B_{11}$$

$$\begin{bmatrix} 9 & 10 \\ 13 & 14 \end{bmatrix} \times \begin{bmatrix} 27 & 28 \\ 31 & 32 \end{bmatrix} = \begin{bmatrix} 545 & 564 \\ 775 & 802 \end{bmatrix}$$

Итерация 2 ($k=1$)

Выбор A:

- C_{00} и C_{01} берут A_{01}
- C_{10} и C_{11} берут A_{11}

Умножения:

$$C_{00}^1: A_{01} \times B_{10}$$

$$\begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \times \begin{bmatrix} 25 & 26 \\ 29 & 30 \end{bmatrix} = \begin{bmatrix} 191 & 198 \\ 407 & 422 \end{bmatrix}$$

$$C_{01}^1: A_{01} \times B_{11}$$

$$\begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \times \begin{bmatrix} 27 & 28 \\ 31 & 32 \end{bmatrix} = \begin{bmatrix} 149 & 156 \\ 293 & 308 \end{bmatrix}$$

$$C_{10}^1: A_{11} \times B_{00}$$

$$\begin{bmatrix} 11 & 12 \\ 15 & 16 \end{bmatrix} \times \begin{bmatrix} 17 & 18 \\ 21 & 22 \end{bmatrix} = \begin{bmatrix} 669 & 692 \\ 901 & 932 \end{bmatrix}$$

$$C_{11}^1: A_{11} \times B_{01}$$

$$\begin{bmatrix} 11 & 12 \\ 15 & 16 \end{bmatrix} \times \begin{bmatrix} 19 & 20 \\ 23 & 24 \end{bmatrix} = \begin{bmatrix} 725 & 748 \\ 977 & 1008 \end{bmatrix}$$

Сложим C_{ij} при $i = \{0,1\}$ и $j = \{0,1\}$ итерации 1 к C_{nm} при $n = \{0,1\}$ и $m = \{0,1\}$ итерации 2. Тогда получаем

$$C_{00} = C_{00}^0 + C_{00}^1 = \begin{bmatrix} 59 & 62 \\ 211 & 222 \end{bmatrix} + \begin{bmatrix} 191 & 198 \\ 407 & 422 \end{bmatrix} = \begin{bmatrix} 250 & 260 \\ 618 & 644 \end{bmatrix}$$

$$C_{01} = C_{01}^0 + C_{01}^1 = \begin{bmatrix} 65 & 68 \\ 233 & 224 \end{bmatrix} + \begin{bmatrix} 149 & 156 \\ 293 & 308 \end{bmatrix} = \begin{bmatrix} 214 & 224 \\ 526 & 552 \end{bmatrix}$$

$$C_{10} = C_{10}^0 + C_{10}^1 = \begin{bmatrix} 515 & 534 \\ 731 & 758 \end{bmatrix} + \begin{bmatrix} 669 & 692 \\ 901 & 932 \end{bmatrix} = \begin{bmatrix} 1184 & 1226 \\ 1632 & 1690 \end{bmatrix}$$

$$C_{11} = C_{11}^0 + C_{11}^1 = \begin{bmatrix} 545 & 564 \\ 775 & 802 \end{bmatrix} + \begin{bmatrix} 725 & 748 \\ 977 & 1008 \end{bmatrix} = \begin{bmatrix} 1270 & 1312 \\ 1752 & 1810 \end{bmatrix}$$

Результат собирается из блоков $C_{00}, C_{01}, C_{10}, C_{11}$. Итог:

$$C = \begin{pmatrix} 250 & 260 & 214 & 224 \\ 618 & 644 & 526 & 552 \\ 1184 & 1226 & 1270 & 1312 \\ 1632 & 1690 & 1752 & 1810 \end{pmatrix}.$$

1.3 Выводы по разделу 1

В данной главе рассмотрены основные понятия, связанные с умножением матриц, проанализированы различные подходы к решению данной задачи, обоснована целесообразность использования параллельных алгоритмов.

Приведены основные параллельные методы умножения матриц и способы их оценки.

На примере небольшой размерности продемонстрированы способы разделения матриц на блоки и алгоритмы их умножения.

Выводы:

- Классический алгоритм прост в реализации, но становится крайне неэффективным при работе с большими матрицами из-за высокой вычислительной сложности и отсутствия параллелизма.
- Оптимизированные последовательные алгоритмы сокращают количество арифметических операций, однако их реализация сложна и не всегда хорошо масштабируется на параллельных архитектурах.
- Параллельные алгоритмы позволяют значительно ускорить процесс умножения за счет эффективного распределения данных и вычислений между несколькими потоками или процессами:
 - Умножение матриц при ленточной схеме разделения данных
 - Алгоритм Фокса
 - Алгоритм Кэннона

2 Программная реализация параллельных алгоритмов умножения матриц

2.1 Технологии параллельного программирования

Параллельное программирование позволяет ускорить вычисления за счёт использования нескольких потоков или процессов. В рамках данной курсовой работы были рассмотрены следующие подходы:

- Многопоточность с использованием стандартной библиотеки `thread` в C++.
- Использование библиотеки `OpenMP` для простого распараллеливания циклов.
- Возможность использования `MPI` (Message Passing Interface) для кластерных систем (однако в данной работе не применялась из-за ограничений среды).

Основное внимание в реализации было уделено библиотеке `OpenMP`, так как она позволяет легко интегрировать параллельное выполнение в существующий код с минимальными изменениями [<https://www.openmp.org/wp-content/uploads/OpenMP-4.5-1115-CPP-web.pdf>].

2.2 Выбор языка и среды программирования

Для реализации алгоритмов параллельного умножения матриц выбран язык программирования C++ с использованием компилятора g++, поддерживающего OpenMP.

Разработка и тестирование программы производились на операционной системе Windows 11 с использованием среды разработки Visual Studio Code и компиляции через WSL (Windows Subsystem for Linux).

Конфигурация системы:

- Процессор: AMD Ryzen 3 4300U with Radeon Graphics 2.70 GHz (4 физических ядра).
- Оперативная память: 8 GB DDR4 3200 MHz.

2.3 Проектирование пользовательского интерфейса

Для исследования эффективности параллельных алгоритмов умножения матриц разработано консольное приложение со следующими функциями:



Рисунок 2.1 – Диаграмма вариантов использования

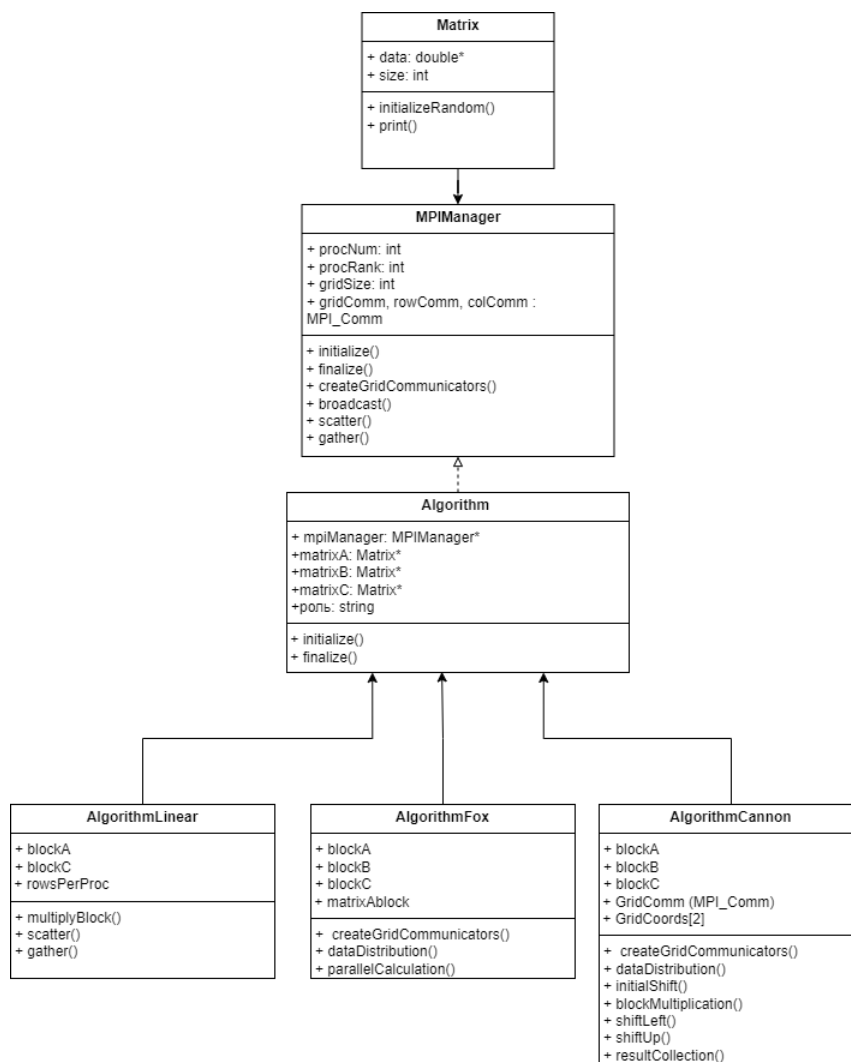


Рисунок 2.2 – Диаграмма классов

2.4 Реализованные функции и алгоритмы

В рамках курсового проекта были реализованы:

1. Алгоритм умножения матриц при ленточной схеме разделения данных.
2. Алгоритм Фокса с блочным разбиением.
3. Алгоритм Кэннона с блочным разбиением.

Каждый из алгоритмов был реализован как отдельное приложение на C++:

- `Algorithm_Linear` – реализация алгоритма умножения матриц при ленточной схеме разделения данных.
- `Algorithm_Fox` – реализация алгоритма Фокса.
- `Algorithm_Cannon` – реализация алгоритма Кэннона.

Входные данные: квадратные матрицы порядка N , заполненные случайными числами от 1 до 10.

2.5 Экспериментальное сравнение параллельных алгоритмов умножения матриц

Тестирование производилось на квадратных матрицах размером 100×100 , 240×240 , 500×500 , 740×740 , 1000×1000 , 2500×2500 и 5000×5000 . Измерялось среднее время выполнения 5 запусков для каждого алгоритма. Результаты представлены в таблице:

Таблица 1 – Результаты экспериментального сравнения алгоритмов при плотности матрицы 25%

	Алгоритм умножения матриц при ленточной схеме разделения данных	Алгоритм Фокса	Алгоритм Кэннона
Время на матрицу 100×100 (мс)	2,6	3	2
Время на матрицу 240×240 (мс)	36	32	33
Время на матрицу 500×500 (мс)	288	201	228
Время на матрицу 740×740 (мс)	892	846	789
Время на матрицу 1000×1000 (мс)	1850	2015	1984
Время на матрицу 2500×2500 (мс)	39477	25788	26803
Время на матрицу 5000×5000 (мс)	757808	260548	278365

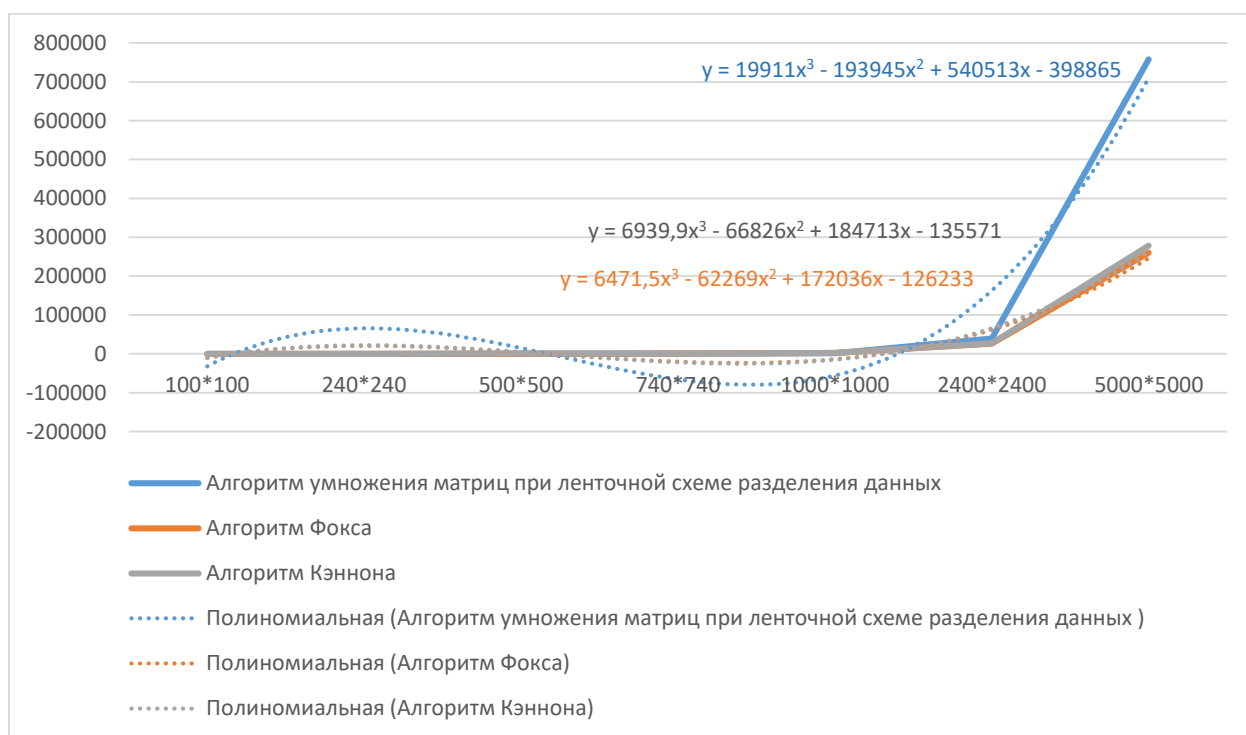


Рисунок 2.3 – Диаграмма сравнения времени выполнения алгоритмов (плотность 25%)

Таблица 2 – Результаты экспериментального сравнения алгоритмов при плотности матрицы 50%

	Алгоритм умножения матриц при ленточной схеме разделения данных	Алгоритм Фокса	Алгоритм Кэннона
Время на матрицу 100×100 (мс)	2,2	2,17	1,2
Время на матрицу 240×240 (мс)	33	34	18
Время на матрицу 500×500 (мс)	291	203	123
Время на матрицу 740×740 (мс)	900	825	484
Время на матрицу 1000×1000 (мс)	1873	2019	1304
Время на матрицу 2500×2500 (мс)	39827	26401	25665
Время на матрицу 5000×5000 (мс)	750441	430046	257230

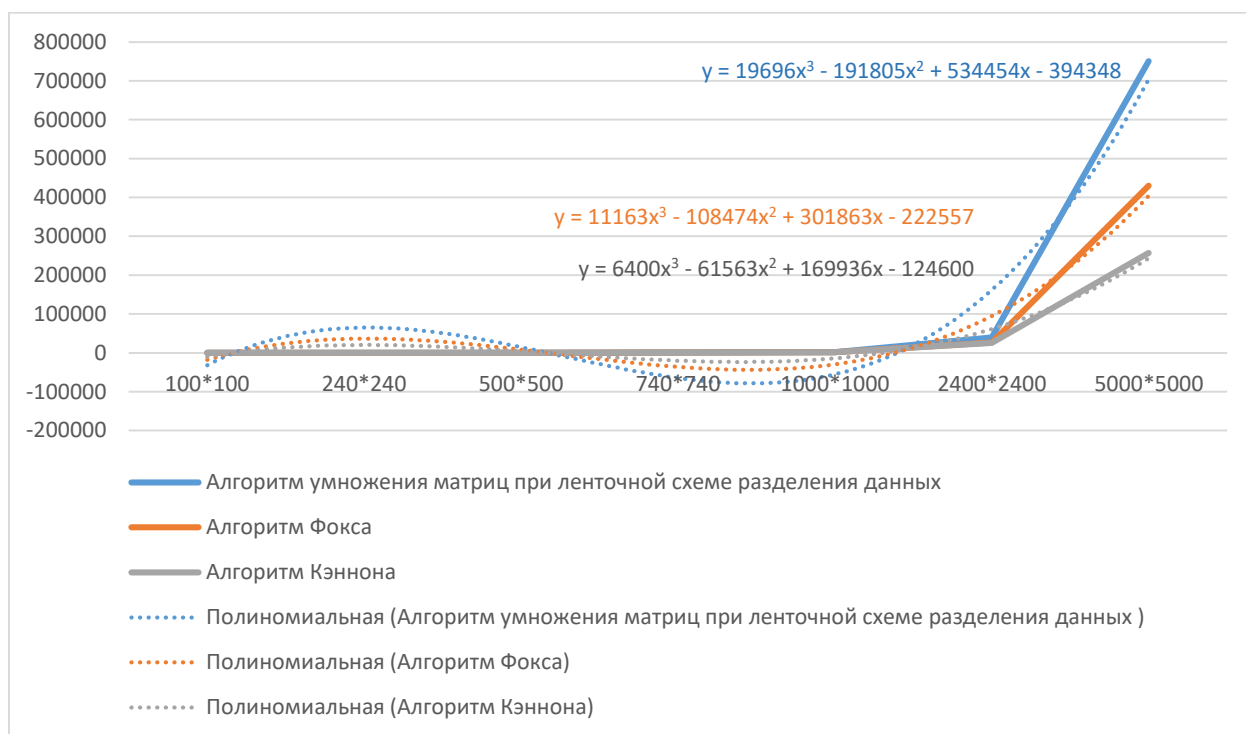


Рисунок 2.4 – Диаграмма сравнения времени выполнения алгоритмов (плотность 50%)

Таблица 3 – Результаты экспериментального сравнения алгоритмов при плотности матрицы 100%

	Алгоритм умножения матриц при ленточной схеме разделения данных	Алгоритм Фокса	Алгоритм Кэннона
Время на матрицу 100 × 100 (мс)	1,6	1,2	1
Время на матрицу 240 × 240 (мс)	34	21	20
Время на матрицу 500 × 500 (мс)	312	123	126
Время на матрицу 740 × 740 (мс)	897	479	465
Время на матрицу 1000 × 1000 (мс)	1882	1297	1304
Время на матрицу 2500 × 2500 (мс)	41189	26438	24819
Время на матрицу 5000 × 5000 (мс)	721640	308849	282049

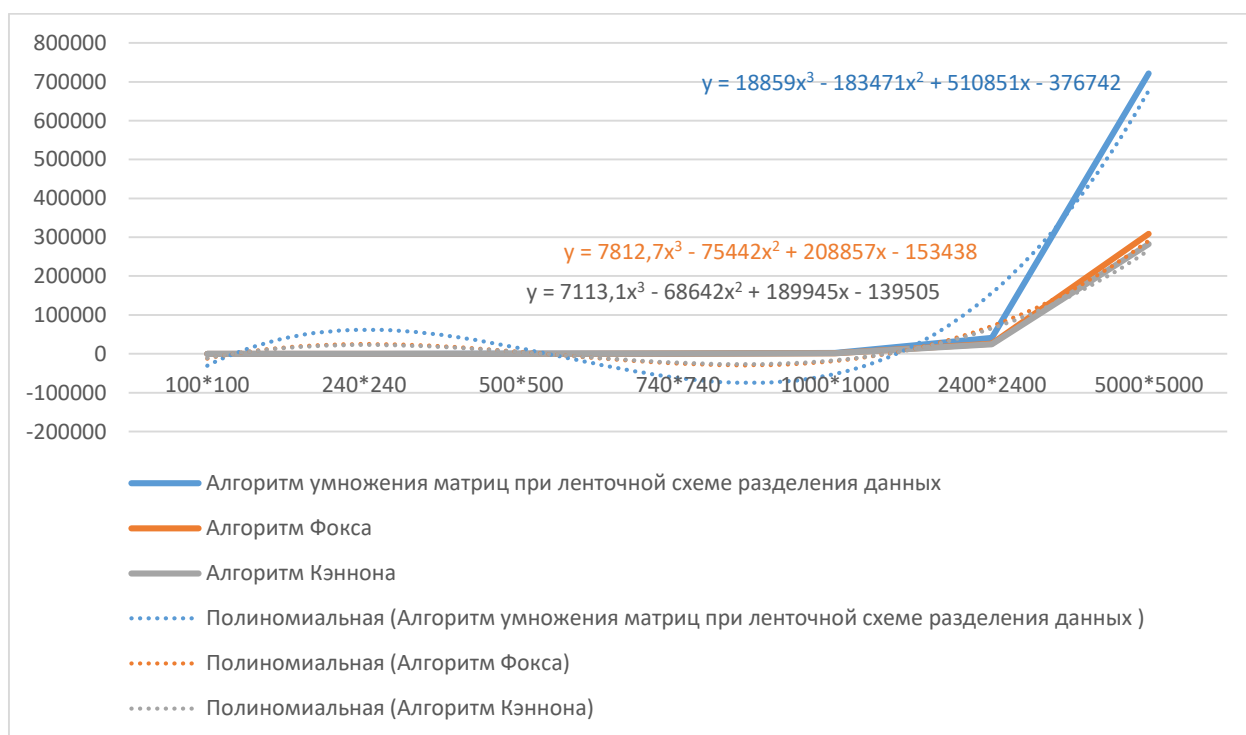


Рисунок 2.5 – Диаграмма сравнения времени выполнения алгоритмов (плотность 100%)

2.6 Выводы по разделу 2

Параллельные алгоритмы умножения матриц позволяют значительно сократить время вычислений на многоядерных системах. Во второй главе были реализованы и протестированы три параллельных алгоритма умножения матриц: ленточный, Фокса и Кэннона. Экспериментальное исследование позволило выявить их поведение на матрицах различной плотности и размера. Результаты показывает следующее:

- Ленточный алгоритм демонстрирует лучшие результаты только на малых матрицах (до 240×240), но проигрывает другим алгоритмам при увеличении размера задачи. Это связано с тем, что он имеет более высокую коммуникационную нагрузку при расширении данных;
- Алгоритм Фокса показывает стабильно хорошую производительность на средних и больших матрицах, особенно при высокой плотности данных. Он лучше масштабируется и легче адаптируется к реальным условиям параллелизма, таким как топология сети и задержки передачи данных;
- Алгоритм Кэннона, хотя и считается теоретически более эффективным по передаче данных, на практике иногда работает медленнее алгоритма Фокса. Это связано с тем, что ему нужны строго определённые условия.

Таким образом, выбор алгоритма зависит от размера данных и доступных вычислительных ресурсов.

Заключение

В рамках курсового проекта:

- Изучены принципы и технологии параллельного программирования.
- Реализованы три алгоритма параллельного умножения матриц.
- Проведено экспериментальное сравнение производительности.

Практическая значимость работы заключается в демонстрации возможностей оптимизации вычислений средствами параллельного программирования в языке C++ и библиотеки OpenMP.

Библиографический список

1. Гергель В.П. Теория и практика параллельных вычислений / В.П. Гергель. – М.: Национальный Открытый Университет «ИНТУИТ», 2016. – 501 с. – (Основы информационных технологий).
2. Крюков В.А. Параллельные вычисления в задачах обработки изображений / В.А. Крюков, А.А. Скороходов // Научно-технический вестник информационных технологий, механики и оптики. – 2015. – № 4 (98). – С. 67–74.
3. Парфенов Н.В. Анализ параллельного алгоритма умножения многомерных матриц // Современные информационные технологии и ИТ-образование. – 2016. – № 12. – С. 45–50.
4. Смирнов А.В. О билинейной сложности и практических алгоритмах умножения матриц // Журнал вычислительной математики и математической физики. – 2013. – Т. 53, № 6. – С. 905–918.
5. Фролов А.Б. Методы распараллеливания алгоритмов на уровне циклов / А.Б. Фролов, А.В. Смирнов // Программная инженерия. – 2018. – Т. 9, № 7. – С. 305–312.
6. Agullo E. Numerical computation with GPUs / E. Agullo, P. Luszczek, J. Kurzak et al. – Cham: Springer, 2014. – 405 p. – ISBN 978-3-319-06547-2. – DOI: 10.1007/978-3-319-06548-9.
7. Buttari A. A class of parallel linear algebra algorithms for multicore architectures / A. Buttari, J. Langou, J. Kurzak, J. Dongarra // Parallel Computing. – 2009. – Vol. 35, No. 1. – P. 38–53. – DOI: 10.1016/j.parco.2008.10.002.
8. Cormen T.H. Introduction to algorithms / T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein. – 3rd ed. – Cambridge: MIT Press, 2009. – 1292 p. – ISBN 0-262-03384-4.
9. Decker E. Parallel algorithms for matrices and graphs / E. Decker, D. Nassimi, S. Sahni // SIAM Journal on Computing. – 1981. – Vol. 10, No. 4. – P. 657–675. – DOI: 10.1137/0210049.

10. Dongarra J.J. Matrix multiplication on a distributed memory multiprocessor / J.J. Dongarra, C.B. Moler, J.R. Bunch et al. // Journal of Computational and Applied Mathematics. – 1990. – Vol. 31, No. 1. – P. 1–14. – DOI: 10.1016/0377-0427(90)90347-K.
11. Goto K. Anatomy of high-performance matrix multiplication / K. Goto, R.A. van de Geijn // ACM Transactions on Mathematical Software. – 2008. – Vol. 34, No. 3. – P. 1–25. – DOI: 10.1145/1356052.1356053.
12. Miller W. Computational complexity and numerical stability / W. Miller // SIAM Journal on Computing. – 1975. – Vol. 4, No. 2. – P. 97–107. – DOI: 10.1137/0204009.
13. Strassen V. Gaussian elimination is not optimal / V. Strassen // Numerische Mathematik. – 1969. – Vol. 13, No. 4. – P. 354–356. – DOI: 10.1007/BF02165411.
14. Vinograd S. On the complexity of multiplying 2×2 matrices / S. Vinograd // Linear Algebra and Its Applications. – 1971. – Vol. 4, No. 4. – P. 381–388. – DOI: 10.1016/0024-3795(71)90009-7.
15. Williams V.V. New bounds for matrix multiplication: from alpha to omega / V.V. Williams, Y. Xu, Z. Xu, R. Zhou // e-print arXiv:2307.07970 [cs.DS], 2024. Bae, S.E.; Shin, T.-W.; Takaoka, T. (2014). «A Faster Parallel Algorithm for Matrix Multiplication on a Mesh Array». Procedia Computer Science . 29: 2230–2240. doi:10.1016/j.procs.2014.05.208

Приложения

Приложение А. Алгоритм умножения матриц при ленточной схеме разделения данных

Файл Algorithm_Linear.cpp:

```
#include <iostream>
#include <cstdlib>
#include <mpi.h>
#include <ctime>
#include <chrono>
using namespace std;
using namespace std::chrono;

// Инициализация случайных данных в матрицах A и B
void RandomDataInitialization(double* pAMatrix, double* pBMatrix, int Size) {
    for (int i = 0; i < Size * Size; i++) {
        pAMatrix[i] = (double)(rand() % 10);
        pBMatrix[i] = (double)(rand() % 10);
    }
}

// Умножение блока A на всю матрицу B, запись в блок C
void MultiplyBlock(double* Ablock, double* B, double* Cblock, int rows, int N) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < N; j++) {
            double sum = 0.0;
            for (int k = 0; k < N; k++) {
                sum += Ablock[i * N + k] * B[k * N + j];
            }
            Cblock[i * N + j] = sum;
        }
    }
}

// Вывод матрицы (как прямоугольник)
void PrintMatrix(double* matrix, int rows, int cols) {
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout.width(7);
            cout.precision(2);
```



```

        cout << fixed << matrix[i * cols + j] << " ";
    }
    cout << endl;
}
}

int main(int argc, char* argv[]) {
    setlocale(LC_ALL, "Russian");
    srand(static_cast<unsigned int>(time(0)));
    int ProcNum, ProcRank;
    int N;
    std::chrono::time_point<std::chrono::high_resolution_clock> start, end;
    double* A = NULL, * B = NULL, * C = NULL;
    double* Ablock = NULL, * Cblock = NULL;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &ProcNum);
    MPI_Comm_rank(MPI_COMM_WORLD, &ProcRank);

    if (ProcRank == 0) cout << "Параллельное умножение матриц (Линейный алгоритм)" <<
endl;
    if (ProcRank == 0) {
        do {
            cout << "Введите размер квадратных матриц (кратный числу процессов " <<
ProcNum << "): ";
            cin >> N;
            if (N % ProcNum != 0)
                cout << "Размер должен быть кратен числу процессов\n";
        } while (N % ProcNum != 0);
    }
    MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
    int rows_per_proc = N / ProcNum;

    // Все процессы выделяют память под B
    B = (double*)malloc(N * N * sizeof(double));
    if (!B) {
        cerr << "Ошибка выделения памяти для B на процессе " << ProcRank << endl;
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    if (ProcRank == 0) {
        A = (double*)malloc(N * N * sizeof(double));

```

```

C = (double*)calloc(N * N, sizeof(double));
if (!A || !C) {
    cerr << "Ошибка выделения памяти на процессе 0\n";
    MPI_Abort(MPI_COMM_WORLD, 1);
}

RandomDataInitialization(A, B, N);

cout << "Матрица A:\n";
PrintMatrix(A, min(6, N), min(6, N));
cout << "Матрица B:\n";
PrintMatrix(B, min(6, N), min(6, N));
start = high_resolution_clock::now();
}

// Выделяем память под блоки
Ablock = (double*)malloc(rows_per_proc * N * sizeof(double));
Cblock = (double*)malloc(rows_per_proc * N * sizeof(double));
if (!Ablock || !Cblock) {
    cerr << "Ошибка выделения памяти под блоки на процессе " << ProcRank << endl;
    MPI_Abort(MPI_COMM_WORLD, 1);
}

// Рассылка данных
if (ProcNum > 1) {
    MPI_Scatter(A, rows_per_proc * N, MPI_DOUBLE,
               Ablock, rows_per_proc * N, MPI_DOUBLE,
               0, MPI_COMM_WORLD);
    MPI_Bcast(B, N * N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
}
else {
    // Если один процесс – просто копируем всё в Ablock
    for (int i = 0; i < N * N; ++i)
        Ablock[i] = A[i];
}

//Параллельное умножение матриц
MultiplyBlock(Ablock, B, Cblock, rows_per_proc, N);

//Сборка результата на процессе 0
if (ProcNum > 1) {
    MPI_Gather(Cblock, rows_per_proc * N, MPI_DOUBLE,
               C, rows_per_proc * N, MPI_DOUBLE,

```

```

        0, MPI_COMM_WORLD);
    }
    else {
        for (int i = 0; i < N * N; ++i)
            C[i] = Cblock[i];
    }
    // Вывод результата
    if (ProcRank == 0) {
        end = high_resolution_clock::now(); //остановка таймера перед выводом
результата
        cout << "Умножение матриц завершено.\n";
        cout << "Результат C (первые 6x6):\n";
        PrintMatrix(C, min(6, N), min(6, N));

        free(A);
        free(B);
        free(C);
        auto duration_milli = duration_cast<milliseconds>(end - start).count();
        cout << "\n\nВремя работы программы: " << duration_milli << " мс" << endl;
    }
    free(Ablock);
    free(Cblock);
    MPI_Finalize();
    return 0;
}

```

Приложение Б. Алгоритм Фокса умножения матриц при блочном разделении данных

Файл Algorithm_Fox.cpp:

```
#include <iostream>
#include <cstdlib>
#include <mpi.h>
#include <ctime>
#include <chrono>
#include <cmath>

using namespace std;
using namespace std::chrono;

// Глобальные переменные
int ProcNum = 0;    // Общее число процессов
int ProcRank = 0;   // Ранг текущего процесса
int GridSize;       // Размер квадратной решетки (число процессов по одной оси)
int GridCoords[2];  // Координаты процесса в решетке
MPI_Comm GridComm;  // Коммуникатор всей решетки
MPI_Comm RowComm;   // Коммуникатор строк решетки
MPI_Comm ColComm;   // Коммуникатор столбцов решетки

// Прототипы функций
void CreateGridCommunicators();
void ProcessInitialization(double*& pAMatrix, double*& pBMatrix, double*& pCMatrix,
    double*& pAblock, double*& pBblock, double*& pCblock,
    double*& pMatrixAblock, int& Size, int& BlockSize);
void RandomDataInitialization(double* pAMatrix, double* pBMatrix, int Size);
void DataDistribution(double* pAMatrix, double* pBMatrix, double* pMatrixAblock,
    double* pBblock, int Size, int BlockSize);
void ABlockCommunication(int iter, double* pAblock, double* pMatrixAblock, int
    BlockSize);
void BlockMultiplication(double* pAblock, double* pBblock, double* pCblock, int
    BlockSize);
void BblockCommunication(double* pBblock, int BlockSize);
void ParallelResultCalculation(double* pAblock, double* pMatrixAblock,
    double* pBblock, double* pCblock, int BlockSize);
void ResultCollection(double* pCMatrix, double* pCblock, int Size, int BlockSize);
void ProcessTermination(double* pAMatrix, double* pBMatrix, double* pCMatrix,
```

```

double* pAblock, double* pBblock, double* pCblock, double* pMatrixAblock);

// Функция для красивого вывода матрицы
void PrintMatrix(double* matrix, int size, int max_rows = 6, int max_cols = 6) {
    for (int i = 0; i < min(size, max_rows); ++i) {
        for (int j = 0; j < min(size, max_cols); ++j) {
            cout.width(7);
            cout.precision(2);
            cout << fixed << matrix[i * size + j] << " ";
        }
        cout << endl;
    }
}

// Создание коммуникаторов в виде двумерной решетки и её подрешеток.
void CreateGridCommunicators() {
    int DimSize[2] = { GridSize, GridSize };
    int Periodic[2] = { 0, 0 };
    int Subdims[2];

    MPI_Cart_create(MPI_COMM_WORLD, 2, DimSize, Periodic, 1, &GridComm);
    MPI_Cart_coords(GridComm, ProcRank, 2, GridCoords);

    Subdims[0] = 0; Subdims[1] = 1;
    MPI_Cart_sub(GridComm, Subdims, &RowComm);

    Subdims[0] = 1; Subdims[1] = 0;
    MPI_Cart_sub(GridComm, Subdims, &ColComm);
}

// Инициализация параметров задачи и выделение памяти
void ProcessInitialization(double*& pAMatrix, double*& pBMatrix, double*& pCMatrix,
    double*& pAblock, double*& pBblock, double*& pCblock,
    double*& pMatrixAblock, int& Size, int& BlockSize) {

    if (ProcRank == 0) {
        do {
            cout << "\nВведите размер матриц: ";
            cin >> Size;
            if (Size % GridSize != 0)
                cout << "Размер матриц должен быть кратен размеру сетки!" << endl;
        }
    }
}

```

```

        } while (Size % GridSize != 0);
    }

    MPI_Bcast(&Size, 1, MPI_INT, 0, MPI_COMM_WORLD);
    BlockSize = Size / GridSize;

    pAblock = new double[BlockSize * BlockSize];
    pBblock = new double[BlockSize * BlockSize];
    pCblock = new double[BlockSize * BlockSize];
    pMatrixAblock = new double[BlockSize * BlockSize];

    for (int i = 0; i < BlockSize * BlockSize; i++)
        pCblock[i] = 0.0;

    if (ProcRank == 0) {
        pAMatrix = new double[Size * Size];
        pBMatrix = new double[Size * Size];
        pCMatrix = new double[Size * Size];
        RandomDataInitialization(pAMatrix, pBMatrix, Size);

        cout << "\nМатрица A (первые 6x6):\n";
        PrintMatrix(pAMatrix, Size);

        cout << "\nМатрица B (первые 6x6):\n";
        PrintMatrix(pBMatrix, Size);
    }
}

// Инициализация матриц A и B случайными значениями
void RandomDataInitialization(double* pAMatrix, double* pBMatrix, int Size) {
    for (int i = 0; i < Size * Size; i++) {
        pAMatrix[i] = (double)(rand() % 10);
        pBMatrix[i] = (double)(rand() % 10);
    }
}

// Распределение матриц по блокам среди процессов
void DataDistribution(double* pAMatrix, double* pBMatrix, double* pMatrixAblock,
    double* pBblock, int Size, int BlockSize) {

    int numBlocks = GridSize;

```

```

if (ProcRank == 0) {
    for (int i = 0; i < numBlocks; i++) {
        for (int j = 0; j < numBlocks; j++) {
            int coords[2] = { i, j };
            int targetRank;
            MPI_Cart_rank(GridComm, coords, &targetRank);

            if (targetRank == 0) {
                for (int ii = 0; ii < BlockSize; ii++) {
                    for (int jj = 0; jj < BlockSize; jj++) {
                        int global_index = (i * BlockSize + ii) * Size + j *
BlockSize + jj;

                        pMatrixAblock[ii * BlockSize + jj] =
pAMatrix[global_index];
                        pBblock[ii * BlockSize + jj] = pBMatrix[global_index];
                    }
                }
            }
            else {
                double* bufferA = new double[BlockSize * BlockSize];
                double* bufferB = new double[BlockSize * BlockSize];

                for (int ii = 0; ii < BlockSize; ii++) {
                    for (int jj = 0; jj < BlockSize; jj++) {
                        int global_index = (i * BlockSize + ii) * Size + j *
BlockSize + jj;

                        bufferA[ii * BlockSize + jj] = pAMatrix[global_index];
                        bufferB[ii * BlockSize + jj] = pBMatrix[global_index];
                    }
                }

                MPI_Send(bufferA, BlockSize * BlockSize, MPI_DOUBLE, targetRank,
0, MPI_COMM_WORLD);
                MPI_Send(bufferB, BlockSize * BlockSize, MPI_DOUBLE, targetRank,
1, MPI_COMM_WORLD);

                delete[] bufferA;
                delete[] bufferB;
            }
        }
    }
}

```

```

    }
}
else {
    MPI_Status status;
    MPI_Recv(pMatrixAblock, BlockSize * BlockSize, MPI_DOUBLE, 0, 0,
MPI_COMM_WORLD, &status);
    MPI_Recv(pBblock, BlockSize * BlockSize, MPI_DOUBLE, 0, 1, MPI_COMM_WORLD,
&status);
}
}

// Рассылка блока матрицы A по строке
void ABlockCommunication(int iter, double* pAblock, double* pMatrixAblock, int
BlockSize) {
    int Pivot = (GridCoords[0] + iter) % GridSize;

    if (GridCoords[1] == Pivot) {
        for (int i = 0; i < BlockSize * BlockSize; i++)
            pAblock[i] = pMatrixAblock[i];
    }

    MPI_Bcast(pAblock, BlockSize * BlockSize, MPI_DOUBLE, Pivot, RowComm);
}

// Умножение блоков матриц
void BlockMultiplication(double* pAblock, double* pBblock, double* pCblock, int
BlockSize) {
    for (int i = 0; i < BlockSize; i++) {
        for (int j = 0; j < BlockSize; j++) {
            double sum = 0.0;
            for (int k = 0; k < BlockSize; k++) {
                sum += pAblock[i * BlockSize + k] * pBblock[k * BlockSize + j];
            }
            pCblock[i * BlockSize + j] += sum;
        }
    }
}

// Циклический сдвиг блока матрицы B по столбцу
void BblockCommunication(double* pBblock, int BlockSize) {
    MPI_Status status;

```



```

int NextProc = (GridCoords[0] + 1) % GridSize;
int PrevProc = (GridCoords[0] - 1 + GridSize) % GridSize;

MPI_Sendrecv_replace(pBblock, BlockSize * BlockSize, MPI_DOUBLE,
    NextProc, 0, PrevProc, 0, ColComm, &status);
}

// Параллельное умножение матриц (алгоритм Фокса)
void ParallelResultCalculation(double* pAblock, double* pMatrixAblock,
    double* pBblock, double* pCblock, int BlockSize) {

    for (int iter = 0; iter < GridSize; iter++) {
        ABlockCommunication(iter, pAblock, pMatrixAblock, BlockSize);
        BlockMultiplication(pAblock, pBblock, pCblock, BlockSize);
        BblockCommunication(pBblock, BlockSize);
    }
}

// Сборка результата на процессе 0
void ResultCollection(double* pCMatrix, double* pCblock, int Size, int BlockSize) {
    int numBlocks = GridSize;

    if (ProcRank == 0) {
        for (int ii = 0; ii < BlockSize; ii++) {
            for (int jj = 0; jj < BlockSize; jj++) {
                pCMatrix[ii * Size + jj] = pCblock[ii * BlockSize + jj];
            }
        }

        for (int proc = 1; proc < ProcNum; proc++) {
            int coords[2];
            MPI_Cart_coords(GridComm, proc, 2, coords);
            double* buffer = new double[BlockSize * BlockSize];
            MPI_Status status;

            MPI_Recv(buffer, BlockSize * BlockSize, MPI_DOUBLE, proc, 2,
MPI_COMM_WORLD, &status);

            for (int ii = 0; ii < BlockSize; ii++) {
                for (int jj = 0; jj < BlockSize; jj++) {
                    int global_i = coords[0] * BlockSize + ii;

```

```

        int global_j = coords[1] * BlockSize + jj;
        pCMatrix[global_i * Size + global_j] = buffer[ii * BlockSize +
jj];
    }
}

    delete[] buffer;
}
}
else {
    MPI_Send(pCblock, BlockSize * BlockSize, MPI_DOUBLE, 0, 2, MPI_COMM_WORLD);
}
}

// Освобождение памяти
void ProcessTermination(double* pAMatrix, double* pBMatrix, double* pCMatrix,
    double* pAblock, double* pBblock, double* pCblock, double* pMatrixAblock) {

    if (ProcRank == 0) {
        delete[] pAMatrix;
        delete[] pBMatrix;
        delete[] pCMatrix;
    }

    delete[] pAblock;
    delete[] pBblock;
    delete[] pCblock;
    delete[] pMatrixAblock;
}

int main(int argc, char* argv[]) {
    setlocale(LC_ALL, "Russian");
    srand(static_cast<unsigned int>(time(0)));
    std::chrono::time_point<std::chrono::high_resolution_clock> start, end;
    double* pAMatrix = NULL, * pBMatrix = NULL, * pCMatrix = NULL;
    double* pAblock = NULL, * pBblock = NULL, * pCblock = NULL, * pMatrixAblock =
NULL;
    int Size, BlockSize;

    setvbuf(stdout, 0, _IONBF, 0);

```

```

MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &ProcNum);
MPI_Comm_rank(MPI_COMM_WORLD, &ProcRank);
if (ProcRank == 0) cout << "Параллельное умножение матриц (алгоритм Фокса)" <<
endl;
GridSize = (int)sqrt((double)ProcNum);
if (ProcNum != GridSize * GridSize) {
    if (ProcRank == 0)
        cout << "Число процессов должно быть полным квадратом!" << endl;
    MPI_Finalize();
    return 1;
}

// Создаем коммуникаторы решетки, строк и столбцов
CreateGridCommunicators();

// Инициализация данных и выделение памяти
ProcessInitialization(pAMatrix, pBMatrix, pCMatrix,
    pAblock, pBblock, pCblock, pMatrixAblock, Size, BlockSize);
if (ProcRank == 0) start = high_resolution_clock::now(); //запуск таймера

// Распределение данных между процессами
DataDistribution(pAMatrix, pBMatrix, pMatrixAblock, pBblock, Size, BlockSize);

// Выполнение параллельного умножения матриц (алгоритм Фокса)
ParallelResultCalculation(pAblock, pMatrixAblock, pBblock, pCblock, BlockSize);

// Сборка результата на процессе 0
ResultCollection(pCMatrix, pCblock, Size, BlockSize);

if (ProcRank == 0){
    // завершение замера времени перед выводом результата
    end = high_resolution_clock::now();
    auto duration_milli = duration_cast<milliseconds>(end - start).count();
    cout << "\nРезультат умножения (первые 6x6):" << endl;
    PrintMatrix(pCMatrix, Size);
    cout << "\nВремя работы программы: " << duration_milli << " мс" << endl <<
endl;
}

// Освобождение памяти

```

```
    ProcessTermination(pAMatrix, pBMatrix, pCMatrix, pAblock, pBblock, pCblock,  
pMatrixAblock);  
  
    MPI_Finalize();  
    return 0;  
}
```

Приложение В. Алгоритм Кэннона умножения матриц при блочном разделении данных

Файл Algorithm_Cannon.cpp:

```
#include <iostream>
#include <cstdlib>
#include <mpi.h>
#include <ctime>
#include <chrono>
#include <cmath>

using namespace std;
using namespace std::chrono;

// Глобальные переменные
int ProcNum = 0;      // Общее число процессов
int ProcRank = 0;     // Ранг текущего процесса
int GridSize;         // Размер квадратной решетки (число процессов по одной оси)
int GridCoords[2];    // Координаты процесса в решетке
MPI_Comm GridComm;    // Коммуникатор всей решетки

// Прототипы функций
void CreateGridCommunicators();
void ProcessInitialization(double*& pAMatrix, double*& pBMatrix, double*& pCMatrix,
    double*& pAblock, double*& pBblock, double*& pCblock, int& Size, int& BlockSize);
void RandomDataInitialization(double* pAMatrix, double* pBMatrix, int Size);
void DataDistribution(double* pAMatrix, double* pBMatrix, double* pAblock, double*
    pBblock,
    int Size, int BlockSize);
void InitialShift(double* pAblock, double* pBblock, int BlockSize);
void BlockMultiplication(double* pAblock, double* pBblock, double* pCblock, int
    BlockSize);
void ShiftLeft(double* pAblock, int BlockSize);
void ShiftUp(double* pBblock, int BlockSize);
void ResultCollection(double* pCMatrix, double* pCblock, int Size, int BlockSize);
void ProcessTermination(double* pAMatrix, double* pBMatrix, double* pCMatrix,
    double* pAblock, double* pBblock, double* pCblock);

// Функция для красивого вывода матрицы
```

```

void PrintMatrix(double* matrix, int size, int max_rows = 6, int max_cols = 6) {
    for (int i = 0; i < min(size, max_rows); ++i) {
        for (int j = 0; j < min(size, max_cols); ++j) {
            cout.width(7);
            cout.precision(2);
            cout << fixed << matrix[i * size + j] << " ";
        }
        cout << endl;
    }
}

// Создание коммуникатора в виде двумерной решетки
void CreateGridCommunicators() {
    int dims[2] = { GridSize, GridSize };
    int periods[2] = { 1, 1 }; // Циклические сдвиги важны для Кэннона
    int reorder = 1;
    MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, reorder, &GridComm);
    MPI_Cart_coords(GridComm, ProcRank, 2, GridCoords);
}

// Инициализация параметров задачи и выделение памяти
void ProcessInitialization(double*& pAMatrix, double*& pBMatrix, double*& pCMatrix,
    double*& pAblock, double*& pBblock, double*& pCblock,
    int& Size, int& BlockSize) {

    if (ProcRank == 0) {
        do {
            cout << "\nВведите размер матриц (кратный " << GridSize << "): ";
            cin >> Size;
            if (Size % GridSize != 0)
                cout << "Размер должен быть кратен " << GridSize << "!\n";
        } while (Size % GridSize != 0);
    }

    MPI_Bcast(&Size, 1, MPI_INT, 0, MPI_COMM_WORLD);
    BlockSize = Size / GridSize;

    pAblock = new double[BlockSize * BlockSize];
    pBblock = new double[BlockSize * BlockSize];
    pCblock = new double[BlockSize * BlockSize];
}

```

```

// Обнуляем результат перед началом
for (int i = 0; i < BlockSize * BlockSize; i++)
    pCblock[i] = 0.0;

if (ProcRank == 0) {
    pAMatrix = new double[Size * Size];
    pBMatrix = new double[Size * Size];
    pCMatrix = new double[Size * Size];
    RandomDataInitialization(pAMatrix, pBMatrix, Size);

    cout << "\nМатрица A (первые 6x6):\n";
    PrintMatrix(pAMatrix, Size);

    cout << "\nМатрица B (первые 6x6):\n";
    PrintMatrix(pBMatrix, Size);
}
}

// Заполнение матриц случайными данными
void RandomDataInitialization(double* pAMatrix, double* pBMatrix, int Size) {
    srand(42); // Для воспроизводимости
    for (int i = 0; i < Size * Size; i++) {
        pAMatrix[i] = (double)(rand() % 10);
        pBMatrix[i] = (double)(rand() % 10);
    }
}

// Распределение блоков матриц по процессам
void DataDistribution(double* pAMatrix, double* pBMatrix,
    double* pAblock, double* pBblock,
    int Size, int BlockSize) {

    int numBlocks = GridSize;

    if (ProcRank == 0) {
        for (int i = 0; i < numBlocks; i++) {
            for (int j = 0; j < numBlocks; j++) {
                int coords[2] = { i, j };
                int rank;
                MPI_Cart_rank(GridComm, coords, &rank);
            }
        }
    }
}

```

```

        if (rank == 0) {
            for (int ii = 0; ii < BlockSize; ii++) {
                for (int jj = 0; jj < BlockSize; jj++) {
                    pAblock[ii * BlockSize + jj] = pAMatrix[(i * BlockSize +
ii) * Size + j * BlockSize + jj];
                    pBblock[ii * BlockSize + jj] = pBMatrix[(i * BlockSize +
ii) * Size + j * BlockSize + jj];
                }
            }
        }
        else {
            double* bufferA = new double[BlockSize * BlockSize];
            double* bufferB = new double[BlockSize * BlockSize];

            for (int ii = 0; ii < BlockSize; ii++) {
                for (int jj = 0; jj < BlockSize; jj++) {
                    bufferA[ii * BlockSize + jj] = pAMatrix[(i * BlockSize +
ii) * Size + j * BlockSize + jj];
                    bufferB[ii * BlockSize + jj] = pBMatrix[(i * BlockSize +
ii) * Size + j * BlockSize + jj];
                }
            }

            MPI_Send(bufferA, BlockSize * BlockSize, MPI_DOUBLE, rank, 0,
MPI_COMM_WORLD);
            MPI_Send(bufferB, BlockSize * BlockSize, MPI_DOUBLE, rank, 1,
MPI_COMM_WORLD);

            delete[] bufferA;
            delete[] bufferB;
        }
    }
}

else {
    MPI_Status status;
    MPI_Recv(pAblock, BlockSize * BlockSize, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
&status);
    MPI_Recv(pBblock, BlockSize * BlockSize, MPI_DOUBLE, 0, 1, MPI_COMM_WORLD,
&status);
}

```



```

}

// Начальный сдвиг блоков для алгоритма Кэннона
void InitialShift(double* pAblock, double* pBblock, int BlockSize) {
    MPI_Status status;
    int src, dst;

    // Сдвиг A влево на GridCoords[0]
    int shiftA = GridCoords[0];
    MPI_Cart_shift(GridComm, 1, -shiftA, &src, &dst);
    MPI_Sendrecv_replace(pAblock, BlockSize * BlockSize, MPI_DOUBLE, dst, 0, src, 0,
GridComm, &status);

    // Сдвиг B вверх на GridCoords[1]
    int shiftB = GridCoords[1];
    MPI_Cart_shift(GridComm, 0, -shiftB, &src, &dst);
    MPI_Sendrecv_replace(pBblock, BlockSize * BlockSize, MPI_DOUBLE, dst, 0, src, 0,
GridComm, &status);
}

// Умножение двух блоков матриц
void BlockMultiplication(double* pAblock, double* pBblock, double* pCblock, int
BlockSize) {
    for (int i = 0; i < BlockSize; i++) {
        for (int j = 0; j < BlockSize; j++) {
            double sum = 0.0;
            for (int k = 0; k < BlockSize; k++) {
                sum += pAblock[i * BlockSize + k] * pBblock[k * BlockSize + j];
            }
            pCblock[i * BlockSize + j] += sum;
        }
    }
}

// Циклический сдвиг блока A влево
void ShiftLeft(double* pAblock, int BlockSize) {
    MPI_Status status;
    int src, dst;
    MPI_Cart_shift(GridComm, 1, -1, &src, &dst);
    MPI_Sendrecv_replace(pAblock, BlockSize * BlockSize, MPI_DOUBLE, dst, 0, src, 0,
GridComm, &status);
}

```

```

}

// Циклический сдвиг блока В вверх
void ShiftUp(double* pBblock, int BlockSize) {
    MPI_Status status;
    int src, dst;
    MPI_Cart_shift(GridComm, 0, -1, &src, &dst);
    MPI_Sendrecv_replace(pBblock, BlockSize * BlockSize, MPI_DOUBLE, dst, 0, src, 0,
GridComm, &status);
}

// Сборка результата на процессе 0
void ResultCollection(double* pCMatrix, double* pCblock, int Size, int BlockSize) {
    if (ProcRank == 0) {
        int myRow = GridCoords[0];
        int myCol = GridCoords[1];

        for (int i = 0; i < BlockSize; i++) {
            for (int j = 0; j < BlockSize; j++) {
                pCMatrix[(myRow * BlockSize + i) * Size + (myCol * BlockSize + j)] =
pCblock[i * BlockSize + j];
            }
        }

        for (int proc = 1; proc < ProcNum; proc++) {
            int coords[2];
            MPI_Cart_coords(GridComm, proc, 2, coords);
            double* buffer = new double[BlockSize * BlockSize];
            MPI_Status status;
            MPI_Recv(buffer, BlockSize * BlockSize, MPI_DOUBLE, proc, 2,
MPI_COMM_WORLD, &status);

            for (int i = 0; i < BlockSize; i++) {
                for (int j = 0; j < BlockSize; j++) {
                    pCMatrix[(coords[0] * BlockSize + i) * Size + (coords[1] *
BlockSize + j)] = buffer[i * BlockSize + j];
                }
            }

            delete[] buffer;
        }
    }
}

```

```

    }
    else {
        MPI_Send(pCblock, BlockSize * BlockSize, MPI_DOUBLE, 0, 2, MPI_COMM_WORLD);
    }
}

// Освобождение памяти
void ProcessTermination(double* pAMatrix, double* pBMatrix, double* pCMatrix,
    double* pAblock, double* pBblock, double* pCblock) {

    if (ProcRank == 0) {
        delete[] pAMatrix;
        delete[] pBMatrix;
        delete[] pCMatrix;
    }

    delete[] pAblock;
    delete[] pBblock;
    delete[] pCblock;
}

int main(int argc, char* argv[]) {
    setlocale(LC_ALL, "Russian");
    srand(static_cast<unsigned int>(time(0)));
    std::chrono::time_point<std::chrono::high_resolution_clock> start, end; //
переменные замера времени
    double* pAMatrix = NULL, * pBMatrix = NULL, * pCMatrix = NULL;
    double* pAblock = NULL, * pBblock = NULL, * pCblock = NULL;
    int Size, BlockSize;

    setvbuf(stdout, 0, _IONBF, 0); // Отключить буферизацию
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &ProcNum);
    MPI_Comm_rank(MPI_COMM_WORLD, &ProcRank);

    if (ProcRank == 0) cout << "Параллельное умножение матриц (алгоритм Кэннона)" <<
endl;

    GridSize = (int)sqrt((double)ProcNum);
    if (ProcNum != GridSize * GridSize) {
        if (ProcRank == 0)
            cout << "Число процессов должно быть полным квадратом!" << endl;
    }
}

```

```

        MPI_Finalize();
        return 1;
    }

    // Создание 2D коммуникаторов
    CreateGridCommunicators();

    // Инициализация данных и выделение памяти
    ProcessInitialization(pAMatrix, pBMatrix, pCMatrix, pAblock, pBblock, pCblock,
Size, BlockSize);
    if (ProcRank == 0) start = high_resolution_clock::now(); // начинаем отсчёт
времени после инициализации

    // Распределение блоков матриц между процессами
    DataDistribution(pAMatrix, pBMatrix, pAblock, pBblock, Size, BlockSize);

    // Начальный сдвиг блоков матриц
    InitialShift(pAblock, pBblock, BlockSize);

    // Основной цикл алгоритма Кэннона
    for (int iter = 0; iter < GridSize; iter++) {
        BlockMultiplication(pAblock, pBblock, pCblock, BlockSize);
        ShiftLeft(pAblock, BlockSize);
        ShiftUp(pBblock, BlockSize);
    }

    // Сборка результата на процессе 0
    ResultCollection(pCMatrix, pCblock, Size, BlockSize);

    // Вывод результата
    if (ProcRank == 0) {
        // остановка таймера перед выводом результата
        end = high_resolution_clock::now();
        auto duration_milli = duration_cast<milliseconds>(end - start).count();

        cout << "\nРезультат умножения (первые 6x6):\n";
        PrintMatrix(pCMatrix, Size);

        cout << "\n\nВремя работы программы: " << duration_milli << " мс" << endl <<
endl;
    }
}

```

```
// Освобождение памяти
ProcessTermination(pAMatrix, pBMatrix, pCMatrix, pAblock, pBblock, pCblock);

MPI_Finalize();
return 0;
}
```